Pondus: an agentic structural analysis and design program

How the agent is set up, how it builds, analyses and designs a structure, and how it was calibrated against benchmarks, SAP2000 and ETABS

Sompote Youwai
AI Research Group, Department of Civil Engineering
King Mongkut's University of Technology Thonburi, Bangkok, Thailand
sompote.you@kmutt.ac.th

October 2026

Abstract

Language-model agents can now operate engineering programs through tool interfaces, but a structural program that an agent drives has to be designed for that user, because the agent accepts whatever numbers the tools return. This paper describes Pondus, an open-source native program in which a language-model agent and an engineer work on one three-dimensional structural model, and reports how it is built, how it is used and how far its results can be trusted. The agent reaches the program through a Model Context Protocol server of 44 tools, either from the chat panel of the desktop application, where a vendored agent harness runs one agent session per message on a copy of the model, or from external agents such as Claude Desktop and ChatGPT. Every tool calls the same functions as the engineer's interface, so a model built by the agent can be checked, edited and undone by hand. The analysis runs on the Stabileo engine, which Pondus links as a library with four marked changes: Pondus splits members, transfers slab loads, generates shear walls as wide columns with rigid arms and builds the seismic mass before handing a solver input to the engine, and it designs to ACI 318-19, AISC 360-16 and EN 1993-1-1 from the results. Two agent runs with a low-cost model built, analysed and designed a two-storey frame in 105 s with 13 tool calls and a five-storey frame with shear walls under an ASCE 7-22 response spectrum, including a 16-page calculation report, in 382 s with 22 tool calls. Every number in the agent's answers matched the tools and hand checks of the loads, the seismic weight and the base shear, but the answers also contained four descriptive errors, such as reporting two deepened beams where the program had deepened six, and they did not point out that the walls resist load only in one direction, so the response modification factor of the walls was applied to the frame direction as well. A calibration against 470 published, closed-form and independent reference values over 229 cases found 430 within 0.5 % and 449 within 1 %, and it found fourteen defects, thirteen of them in the code that prepares the inputs of the engine and the design checks rather than in the numerical kernels. A second calibration against the verification problems that CSI publishes for SAP2000 and ETABS reproduced 177 of 199 independent values within 0.5 % and found eight further defects, including an unconservative omission of the Eurocode 3 moment reduction for high shear, which led to lumped masses, the SRSS combination and frame shear deformation being added. Together these results suggest that the tools of an agentic structural program can be made accurate, whereas the agent's prose remains the weaker link that the engineer must read against the program's own tables.

Keywords agentic AI; language-model agents; Model Context Protocol; structural analysis; structural design; Stabileo; ACI 318; verification

1 Introduction

Structural engineers have used analysis software for decades, but the way they operate it is changing because language-model agents can now plan a sequence of actions and call software tools to carry them out [1, 2]. The Model Context Protocol (MCP) [3] has made this practical, because a program that exposes its operations as MCP tools can be driven by any agent that speaks the protocol. An engineer can therefore type a request such as "a three-storey concrete frame, three bays of 6 m by two bays of 5 m, design the beams and slabs" and watch an agent build the model, analyse it and size every member in a minute or two. The speed is real, but so is the risk, because an agent has no habit of distrusting a number that looks plausible and it can describe a result incorrectly even when the number is right [4].

Pondus [5] was written to explore what a structural program should look like when an agent is one of its users. Its design follows three principles. The agent and the engineer act on one model through the same commands, so everything the agent does can be inspected, edited and undone in the ordinary interface. The tools return the checks an engineer would make, such as the balance of applied loads and reactions and the share of slab load that reached the beams, so that the agent is prompted to verify its own work. The analysis engine is an existing, verified open-source kernel, Stabileo [6], so the effort of the project goes into the layer above it, which turns a building model into solver inputs and solver outputs into design decisions.

This paper describes that program for both its audiences. Section 2 gives the architecture, Section 3 explains how the agent is set up inside the application and from external clients, Sections 4 and 5 explain how the agent builds, analyses and designs a structure, and Section 6 explains how Pondus connects to the Stabileo engine and to the Stabileo and Aedifex model files. Section 7 reports two recorded agent runs and audits the agent's answers against the program and against hand calculations. Section 8 summarises the calibration of the program against published benchmarks, which a companion report gives in full [7], and Section 9 discusses what the results mean for agent-driven engineering software.

2 Architecture

Pondus is a native Rust program organised in four crates, shown with the agents around them in Figure 1. The engine crate is the Stabileo analysis engine, `dedaliano-engine` at commit `7985e8c`, vendored into the workspace so that one build produces the whole program. The core crate holds the model, the 23 commands that build and edit it, the analysis wrapper, the design codes, the advanced analyses, the calculation report and the exchange formats, and it also contains the MCP server. The application crate draws the window with its ribbon, tables, three-dimensional view and chat panel, and the fourth crate is the TigrimOS agent harness [8], a command-line agent (`tigrim`) that the chat panel starts for each message. The same binary therefore runs as a desktop application, as an MCP server (`pondus --mcp`) and as a batch designer (`pondus design in out`).

The model is deliberately close to the way an engineer describes a building rather than to a finite element mesh. A member is one straight piece between two nodes with a kind (beam, column or brace), a section, a material, end releases and design attributes such as the unbraced length. A slab is a rectangular panel resting on the beams under its edges, a wall is a wall element with a thickness, a plan line and a height range, and load cases, combinations, supports, diaphragms, moving loads, construction stages and the seismic settings complete the description. The analysis mesh, the rigid wall arms and the slab shell elements are generated from this description at each analysis and never edited directly, so neither the agent nor the engineer can leave the model and its mesh inconsistent.

Figure 2 shows the window with a portal frame drawn by hand and the agent panel open on the right. The engineer can draw members, bays, slabs and walls with the mouse, edit any value in the tables or ask the agent in the chat, and the three routes produce the same model, so the work can pass from one to the other at any time. The status line reports, after every analysis, whether the reactions balance the applied loads and how much slab load reached the beams, which is the same check the agent receives from its `analyze` tool.

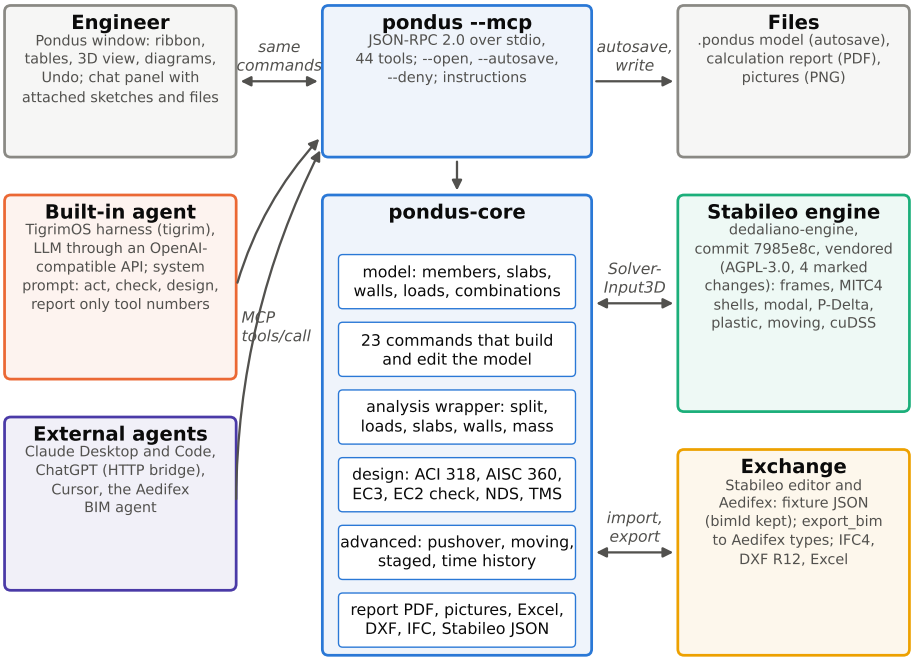


Figure 1: Architecture of Pondus. The engineer's window and every agent reach the same commands of `pondus-core`; agents do so through the MCP server. The core hands a solver input to the vendored Stabileo engine, saves the model and the report for the engineer, and exchanges models with the Stabileo editor, the Aedifex BIM modeller and other programs.

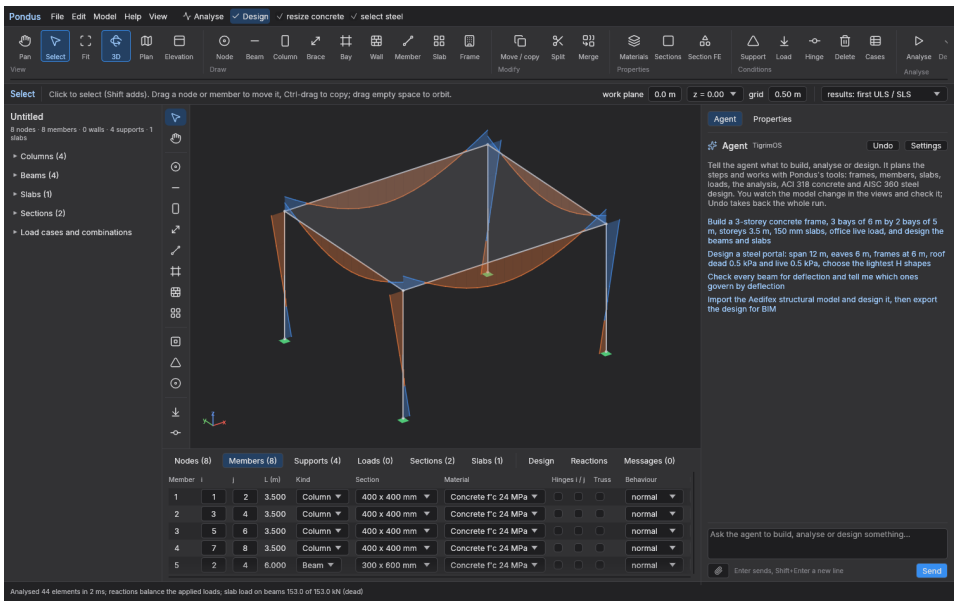


Figure 2: The Pondus window: ribbon (top), drawing palette (left), the model with its bending moment diagram, the member table (bottom) and the agent panel with example requests (right). The status line reports the analysis time and the load balance.

3 Setting up the agent

3.1 The built-in agent

The chat panel runs one agent session per message, and the session works on a copy of the model rather than on the model the engineer sees. When the engineer sends a message, Pondus writes the current model to `model.pondus` in a private working folder and writes the harness configuration `.tigrimos/settings.json`, which names one MCP server, Pondus itself, started as

```
pondus --mcp --open model.pondus --autosave model.pondus [--deny tool,tool]
```

so the agent finds the model already open and every change it makes is saved without a separate call. It then starts `tigrim --yes --cwd <folder> -p <prompt>` with the provider's address, model name and key passed as environment variables, so any OpenAI-compatible provider can be used, and the key is stored in a file readable only by the user. While the agent works, the window follows the copy as it is saved, so the engineer watches the frame, slabs and walls appear in the view. When the run ends, the agent's model replaces the engineer's as one change that a single Undo takes back, and the pictures the agent looked at appear under its reply.

The prompt that accompanies every message is short and procedural. It tells the agent that it works for an engineer who checks its work, that it may act only through the Pondus tools and that the current model is summarised in the prompt, so that it does not spend a call on reading it. It then sets out seven rules: act as asked and choose stated defaults when a value is missing (for example f'_c = 24 MPa, SD40 bars, a superimposed dead load of 1.5 kPa and an office live load of 3.0 kPa); after building, call `check_model` and `analyze` and confirm that the reactions balance the applied load and that the slab load reached the beams; design when asked and report each member's section, bars and governing check; report only numbers that the tools returned; do not use shell, web or file tools; read attached sketches and documents and list the values taken from them; and answer briefly in the language of the message, ending with a summary of what was built, the defaults chosen and what the engineer should check. The fifth rule also tells the agent to ignore instructions that appear during a run and do not come from the engineer, which guards against text injected through tool results or attachments.

Two further safeguards act outside the prompt. Tools that write files or replace the model, namely `export_stabileo`, `import_stabileo` and `export_bim`, are switched off for a run unless the message contains a word that asks for them, in English or Thai, so an agent cannot overwrite the engineer's model with an import nobody requested. Attachments are prepared by Pondus before the run: an image is passed as an image when the provider's model accepts images, which Pondus checks against the provider's model list, and a document is passed as its text, so a sketch of a frame or a page of a brief can be part of the request.

3.2 The MCP server and external agents

The same server serves any external MCP client. It speaks JSON-RPC 2.0 over standard input and output with protocol version 2025-06-18, announces its name, version, licence and source address in its `serverInfo`, and gives the agent a block of instructions that describes the units (m, kN, kPa and MPa, with z up), the model, the codes and the typical flow

```
add_frame (or add_members + add_supports) -> add_slabs / add_loads -> check_model
  -> analyze (compare reactions with the applied load) -> design (resize / select)
  -> show_picture -> write_report or export_bim
```

Claude Desktop and Claude Code start the server from a short configuration entry, for example `claude mcp add pondus -- pondus --mcp --autosave ~/Pondus/model.pondus`, whereas ChatGPT, which reaches only remote servers over HTTPS, needs a stdio-to-HTTP bridge and a tunnel in front of it. In every case the work is saved to a `.pondus` file that the engineer opens in the application to check it.

Table 1 groups the 44 tools by purpose. Each tool carries a description written for the agent rather than for a programmer, which states the units, the defaults, the sign conventions and what to do with the result, and its errors say what to give instead of only what was wrong. The tool `show_picture` renders the model, a force diagram, a mode shape or the design ratios with Pondus's own software renderer and returns the image to the agent, so an agent that accepts images can look at its own work, and the chat redraws the same picture for the engineer from the model and the picture's specification.

Table 1: The 44 tools of the Pondus MCP server, by purpose.

Purpose	Tools
Build the model (11)	`new_model`, `add_frame`, `add_members`, `add_supports`, `add_slabs`, `add_walls`, `add_material`, `add_load_case`, `add_combination`, `add_loads`, `set_diaphragms`
Edit the model (9)	`set_members`, `set_material`, `set_walls`, `move_node`, `transform`, `split_member`, `merge_members`, `delete`, `set_design`
Inspect (3)	`get_model`, `list_sections`, `check_model`
Analyse (11)	`analyze`, `get_results`, `get_slab_results`, `set_seismic`, `get_seismic`, `buckling`, `time_history`, `pushover`, `set_moving`, `influence_line`, `set_stages`
Analyse, continued (2)	`staged`, `section_analysis`
Design and report (3)	`design`, `show_picture`, `write_report`
Exchange (5)	`export_file` and `import_file` (Excel, DXF, IFC), `import_stabileo`, `export_stabileo`, `export_bim`

4 How the agent builds and analyses a structure

The agent builds a regular building with one call. The tool `add_frame` takes the bay widths in x and y, the storey heights, the column and beam sections and an optional slab specification, and it creates columns at every grid intersection, beams on every grid line at every floor, supports under the columns and a slab panel in every bay, with the columns' strong axis turned towards the longer spans. Irregular structures are built member by member with `add_members`, floors with `add_slabs` and shear walls and cores with `add_walls`, which the instructions ask the agent to call after the floors so that each wall receives a joint at every floor it passes. The section names follow the engineer's vocabulary, such as `"300 x 600"` for a concrete rectangle in millimetres and `"H 300x150x6.5x9"` for a JIS rolled shape, whose properties Pondus computes from the plate dimensions and fillets.

The tool `check_model` and the start of every analysis look for the mistakes that make a model wrong without making it fail, such as orphan nodes, slab edges that no beam supports, members of zero length and missing supports. The analysis then proceeds in the stages of Figure 3. Each member is split at every node that lies on it, at the ends of every partial load and at the mesh points of plate slabs, and into at least eight elements for a beam and two for a column, so every element carries a load that is linear along its length and every result station falls on an element node. Slab loads go to the edge beams along the 45-degree lines of each panel, or one way when the aspect ratio exceeds two, or the slabs are meshed into MITC4 shell elements [9] that share nodes with the frame. Each wall becomes a wide column [10]: a pier on the wall's centre line with the wall's section and a shear area of $5A/6$, joined at every level by rigid arms to the wall's ends, where beams and floor diaphragms connect. The arms take the wall's own section and 1000 times its modulus, a choice that the calibration forced after an absolute arm stiffness lost precision in tall wall-frame models (Section 8).

The engine factors the stiffness matrix once and solves every load case on it, and a combination is the factored sum of its cases, which is exact for a linear analysis. Pondus then reads the member forces at every station, maps them to its own conventions, with sagging moments and tension positive, and returns

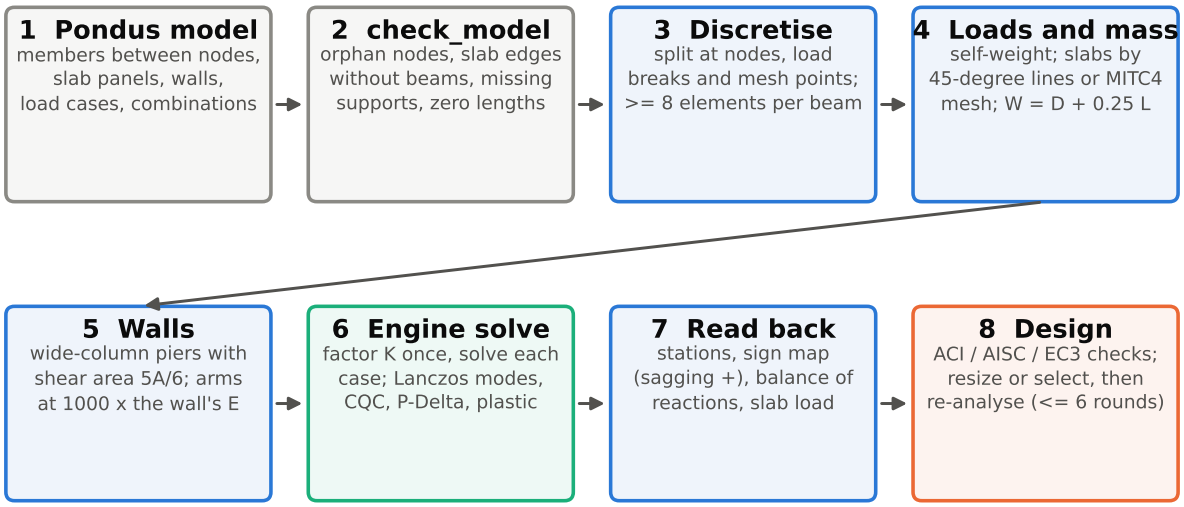


Figure 3: From the Pondus model to the engine and back. Every analysis rebuilds the engine input from the model, so the mesh, the slab loads, the wall arms and the seismic mass always follow the current model.

to the agent a compact summary rather than the raw results: per case the applied load and the reactions and whether they balance, the slab load placed on the beams against the slabs' total, per combination the largest displacement, moment and compression with the member where they occur, and any warnings. The summary is written so that the checks the prompt asks for can be read directly from it.

The seismic analysis follows ASCE 7-22 Chapter 12 [11] and the Thai standards DPT 1301/1302-61 [12] that adopt it. The seismic weight is $W = D + 0.25L$ from the model's own load cases, converted into element mass densities so that the engine's consistent mass carries it. The modes are extracted until 90 % of the mass takes part in each horizontal direction and combined by CQC [13] at every station, the response spectrum base shear is scaled up to the equivalent lateral force value with $T \le C_u T_a$, and the storey drifts $C_d \delta_e / I_e$ are checked against the allowable drift. The advanced analyses, namely second-order P-Delta and linear buckling, pushover by successive plastic hinges, moving-load envelopes and influence lines, construction stages and linear time history under a recorded ground motion, are separate tools that run on the same model.

5 How the agent designs a structure

The `design` tool designs every member type from the analysis results. Concrete beams are designed to ACI 318-19 [14] per third of the span, with top bars from hogging and bottom bars from sagging moments, stirrups for the end quarters and the middle half, torsion reinforcement when compatibility torsion from plate slabs exceeds the threshold, and deflection from the effective inertia with the long-term multiplier. Two-way slabs on beams use the coefficients of Method 2 of ACI 318-63 [15], which the Thai standard EIT 1008 adopts, one-way slabs use the coefficients of ACI 318-19, and plate slabs take their bars from the element moments by the Wood–Armer rules [16, 17]. Concrete columns get the least bar arrangement that passes axial load and biaxial bending by Bresler's load contour [18], walls are designed storey by storey to Chapters 11 and 18.10, and spread footings are sized under the concrete columns from the allowable bearing pressure. Steel members follow AISC 360-16 [19] or EN 1993-1-1 [20], and timber, masonry and cold-formed members follow their

own standards.

With `resize`, failing concrete beams are deepened in 50 mm steps and failing slabs are thickened, and with `select` each steel member or group receives the lightest passing shape. A changed section changes the stiffness and therefore the forces, so the model is analysed again after every change until no section changes, for at most six rounds. The design result returned to the agent lists for each member its section, bars, stirrups or shape, its ratio and the governing check, and it lists separately every failing member, every member that was skipped with the reason, and every change that resizing made. The tool `write_report` turns the same results into an A4 calculation report with the basis of design, the load balance, the diagrams, the earthquake tables, every design table with failing rows in red and the full calculation of the governing member of each type (Figure 4).

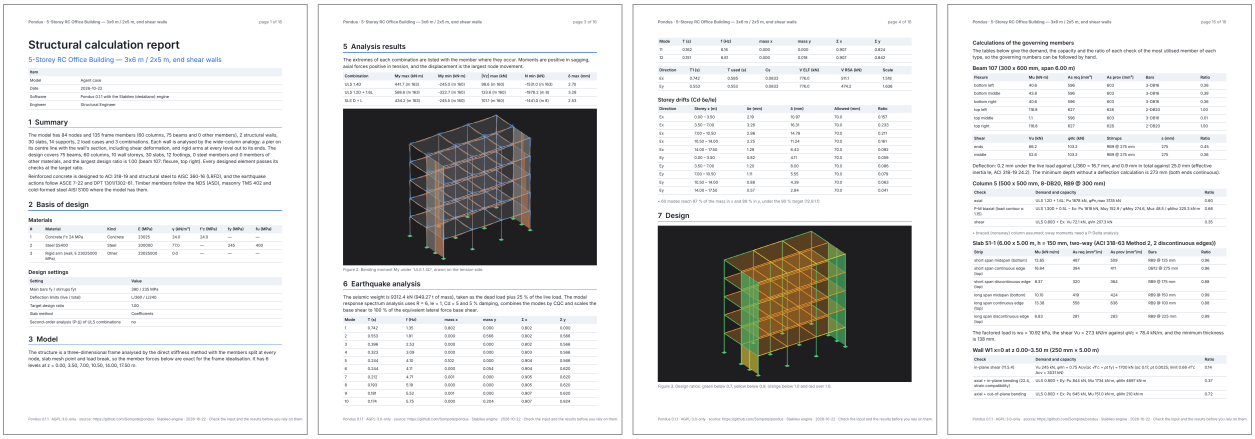


Figure 4: Four of the sixteen pages of the calculation report that the agent wrote in run 2 (Section 7) with `write_report`: the summary and basis of design, the analysis results with the moment diagram, the earthquake analysis and design ratios, and the calculation of the governing members.

6 Connecting to Stabileo

6.1 The engine as a library

Stabileo is an open-source structural analysis program by LambdaClass whose engine, written in Rust, is compiled to WebAssembly for its browser editor [6]. Pondus compiles the same engine natively and calls it as a library, with the manifest trimmed to a native library and four source changes, each marked `[Pondus]` in the code and listed in the program's notice as the AGPL-3.0 licence asks. The first corrects the sign of the w–θ_y coupling in the vertical Winkler foundation matrix, without which a uniformly loaded beam on a uniform foundation settled unevenly. The second tries a sparse Cholesky factorisation before the dense rank test that the plastic solver runs at every hinge event, which gives the same answer and cut the pushover of a three-storey test building from 28 s to 3.7 s. The third keeps the moving-load envelope at each end of an element, because the per-element extreme carried the value one element away from the point asked for. The fourth adds a GPU factorisation through NVIDIA's cuDSS, loaded at run time so that the same binary runs without it, together with a parallel Lanczos reorthogonalisation that reduced the earthquake analysis of a 41,000-element tower from 290 s to 60 s on a 16-core CPU.

The interface between the two programs is the engine's `SolverInput3D` structure, which holds nodes, materials, sections, elements, supports, loads, constraints and shell elements keyed by identifier. Pondus fills it from its own model at every analysis, and the conventions it has to respect were measured in tests rather than assumed: the modulus is given in MPa, the start and end of a distributed load are distances in metres from node i, a column with roll zero bends about its strong axis in the x–z plane, the quad pressure acts along

the element normal so that a downward load is negative, and the plate moments come out hogging-positive. Results return through the engine's diagram evaluation at Pondus's stations. Because the engine is a library and not a separate process, Pondus can also call its specialised solvers directly, namely the P-Delta, modal, plastic, moving-load, contact, cable and Winkler solvers, while keeping its own implementation where the engine's did not fit a design program, as for construction stages, whose engine solver omitted the fixed-end forces from the member forces.

6.2 Model exchange with the Stabileo editor and Aedifex

Pondus also reads and writes Stabileo's model file, the fixture JSON of the web editor, which is also what the Aedifex BIM modeller writes with its `export_stabileo` tool. On import, chains of collinear elements of one section joined at nodes that nothing else uses are merged back into one design member, because Aedifex splits each beam into four, and each element's `bimId` is kept. On export, the model is written as nodes, elements, supports and loads in Stabileo's conventions. The design goes back to the BIM model through `export_bim`, which returns for each member its BIM identifier, its ends, the Aedifex family and type with its parameters, the reinforcement and the ratio, so that an agent can apply it in Aedifex with `duplicate_type` and `change_type`.

The exchange was tested by a round trip on the two models of Section 7: each model was exported to Stabileo JSON, imported into an empty Pondus session and analysed again. For the two-storey frame the reactions and the governing moments and axial forces of all three combinations were reproduced exactly. For the five-storey frame with walls the reactions were again exact, whereas the governing moment of 568.61 kN m in the first-storey wall pier became 568.33 kN m (−0.05 %) and the largest column compression changed by less than 0.01 %. The small differences arise because Stabileo's file has no wall or slab objects, so the walls travel as their piers and arms and the slabs as the beam loads they produce, and the re-imported model therefore carries no wall or slab design. The Stabileo format is thus suitable for passing a frame to the Stabileo editor or receiving one from Aedifex, while a model with walls and slabs should travel as a `.pondus` file or as IFC4.

7 Two recorded agent runs

7.1 Set-up

Two requests were sent through the same path as the chat panel, by a script that writes the working folder and the harness configuration exactly as the application does and starts `tigrim` with the same prompt. The provider was OpenRouter and the model was `deepseek-v4.1-flash`, a low-cost model chosen to represent the least capable agent a user might configure. Pondus 0.1.1 ran on a MacBook (macOS 14, CPU only). The progress output of the harness was recorded with time stamps, and after each run the saved model was opened in a fresh Pondus session, analysed and designed again, so that every statement in the agent's answer could be compared with the program's own output and with hand calculations.

Run 1 asked: "Build a two-storey reinforced concrete frame: 2 bays of 6 m along x and 1 bay of 5 m along y, storeys 3.5 m, columns 400×400, beams 300×600, 150 mm slabs with 1.5 kPa finishes and 3 kPa office live load (roof 1 kPa). Analyse it, check the reactions, then design the beams and slabs and tell me the reinforcement of the most loaded beam." Run 2 asked for a five-storey office building of three bays of 6 m by two bays of 5 m with 500×500 columns, 300×600 beams and the same floors, with two 250 mm shear walls 5 m long on the end grid lines $x = 0$ and $x = 18$ m between $y = 0$ and $y = 5$ m, an ASCE 7-22 spectrum with $S_{DS} = 0.5$ g, $S_{D1} = 0.3$ g and $T_L = 6$ s for special reinforced concrete walls ($R = 6$, $C_d = 5$, $\Omega_0 = 2.5$), a drift check, a design of the beams, slabs, columns, walls and footings with resizing, pictures of the first mode and the design ratios, and a calculation report.

7.2 What the agent did

Figure 5 shows the tool calls of both runs against time. In run 1 the agent built the frame with one `add_frame` call after 6 s, checked and analysed it, and spent the remaining 90 s on seven `design` calls with different subsets of members and one `get_results`, `get_model` and `show_picture` call each, finishing in 105 s with 13 tool calls. In run 2 the agent built the frame, added both walls with one `add_walls` call, set the seismic parameters and analysed the building within 9 s, and the analysis with the modal and response spectrum stages took 13 s. Most of the 382 s then went to eight `design` calls of 10–30 s each, before two pictures, the report and a final analysis, for 22 tool calls in total. In both runs the model was complete after the first few seconds and the time was spent on design, which shows that the agent's habit of re-running the design on subsets of members to read their details, rather than the analysis, sets the duration of a run.

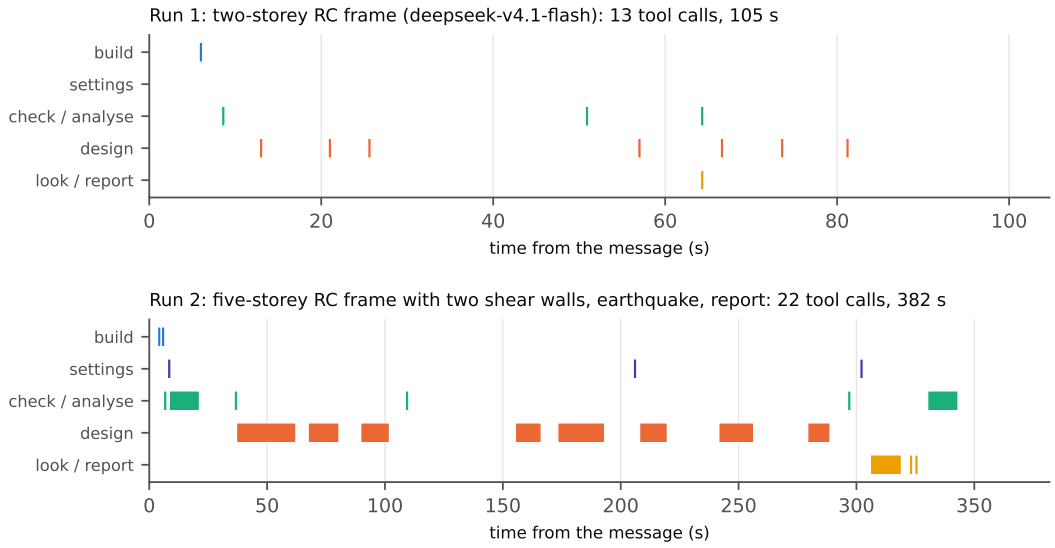
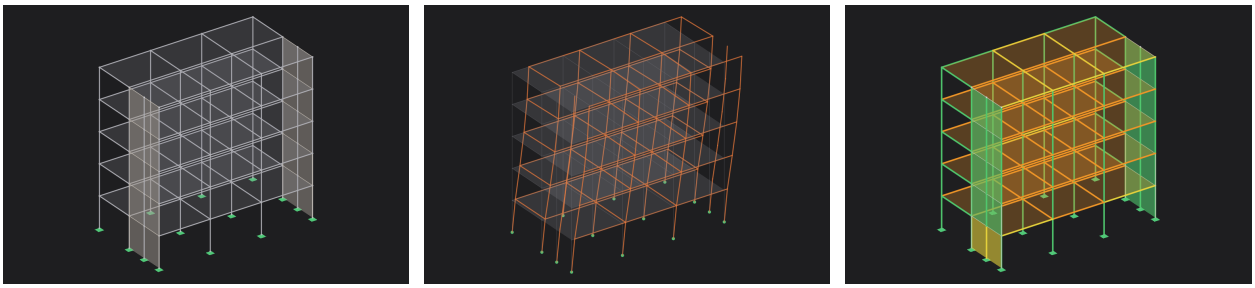


Figure 5: Tool calls of the two agent runs against time, grouped by purpose. Each bar spans a call from its start to its result; gaps are the language model's own time between calls.

The run 2 model has 60 columns, 75 beams, 30 slab panels and two walls of five piers each with 24 rigid arms, with the beams that lay inside the walls replaced by them, as Figure 6 shows. The agent set $C_t = 0.0488$ and $x = 0.75$ for the approximate period, the value of ASCE 7-22 Table 12.8-2 for "all other structural systems", which is the right row for a building frame with shear walls, and it kept the live load in the seismic combinations at 0.5. Resizing deepened six end-bay beams to 300×650 mm, four on grid line $y = 5$ m that frame into the wall ends at the first two floors and two on grid line $y = 10$ m at the second floor, and the final design has no failing member. One step deserves comment: midway through the design the agent switched off the floor vibration screen (`set_design vibration_hz 0`) and switched it back on 96 s later. A design made afterwards with the default settings has no failing member, so the step did not change the outcome, but an agent that changes a design setting during a run should report it, and this one did not.

7.3 Audit of the answers

Table 2 compares the statements of the two answers with the program's output and with hand calculations. The load totals are exact: for run 1 the slabs give $(0.15 \times 24 + 1.5) \times 120 = 612.0$ kN, the 78 m of beams $78 \times 0.3 \times 0.6 \times 24 = 337.0$ kN and the 42 m of columns 161.3 kN, a total of 1110.24 kN, which the agent reported with the matching reactions. For run 2 the slabs give 4590 kN, the frame members 3087.4 kN and the walls 1050 kN, a total of 8727.36 kN, again as reported. The seismic quantities of run 2 also follow by hand: $T_a = 0.0488 \times 17.5^{0.75} = 0.418$ s, $C_s = S_{DS}/(R/I_e) = 0.0833$, which governs because the upper limit



(a) model **(b)** mode 1, T_1 = 0.742 s **(c)** design ratios

Figure 6: Run 2 as Pondus draws it with `show_picture`: (a) the five-storey frame with the two end walls, (b) the first mode in x, in which the frames carry the building and the walls bend out of their plane, and (c) the design ratios after resizing (green below 0.7, yellow below 0.9, orange below 1.0).

Table 2: Statements in the agent's answers against the program's output and hand checks.

Run	Agent's statement	Program output or hand check
1	D = 1110.24 kN, L = 240.0 kN, balanced	exact by hand; reactions equal
1	beam 8: top 3-DB16, bottom 3-DB16, RB9 @ 275, ratio 0.756	as in the design output
1	column ratio 0.451 (P_u 96 kN) is an "interior first-storey" column	the value belongs to the four corner columns of the second storey (members 14, 15, 18, 19)
1	"footing design output was not returned"	six footings were designed, ratios 0.95–0.99
2	D = 8727.4 kN, L = 2340 kN, W = 9312 kN	exact by hand
2	T_a 0.418 s, C_s 0.0833, V_{ELF} 776.0 kN, drifts within 70 mm	exact by hand; drift ratio 0.233 as output
2	beams 16 and 18 deepened to 300 × 650	six beams were deepened (16, 18, 45, 47, 48, 50)
2	columns ratios 0.53–0.66	0.33–0.66
2	wall W2 "identical by symmetry" to W1	first-storey ratios 0.72 (W1) and 0.65 (W2)
2	mode 1: "the two end shear walls translate together in x"	the walls lie along y; mode 1 is the frame mode in x with the walls bent out of plane

$S_{D1}/(TR/I_e)$ at $C_uT_a = 0.585$ s is 0.0855, $W = 8727.4 + 0.25 \times 2340 = 9312.4$ kN and $V = 776.0$ kN, all as reported, and the allowable storey drift is $0.020 \times 3500 = 70$ mm.

Every number in the two answers appears in a tool result, so the rule to report only the tools' numbers held. The errors lay in the words around the numbers: a correct ratio was attached to the wrong column, a correct list of design changes was cut short, a range was quoted from a subset of members, a symmetry was asserted that the results did not show and the first mode was explained by the wrong mechanism. Run 1 also told the engineer that the footings had not been designed when the program had designed them. None of these errors would change the structure as built, because the model and the report are the program's, but each would mislead an engineer who read only the chat.

The more important omission concerns engineering judgement. The walls lie along y, so they resist the earthquake only in that direction, whereas in x the building is a moment frame whose walls bend out of their plane, as the first mode shows. Pondus applies one set of seismic factors to both directions, and the agent applied the factors of special reinforced concrete walls, $R = 6$ and $C_t = 0.0488$, to the frame direction as well. An ordinary or intermediate concrete moment frame takes $R = 3$ or 5 under ASCE 7-22 Table 12.2-1, so the design forces in x would be 1.2 to 2 times larger, and the period coefficients of a concrete moment frame ($C_t = 0.0466$, $x = 0.9$) would also apply. The request named the wall factors, so the agent followed the instruction, but an experienced engineer would have asked about the lateral system in x before designing. The case also exposes a gap in the program, because a tool that accepted separate factors for each direction, or that warned when a direction contains no element of the named system, would have put the question in front of the agent.

8 Calibration

8.1 Method

An agent cannot notice that a tool is wrong, so the tools themselves were calibrated against reference values with known answers. Each comparison is a Rust integration test that builds a Pondus model or calls a Pondus design function, the same functions the design loop uses, and compares the result with a value printed in a textbook, a software verification manual or a published worked example of a design code, with a formula of such a source evaluated in the test, or with an exact solution by an independent method. No comparison calls the engine's checkers directly, because the engine had been verified by its authors and the risk lay in the layer that Pondus adds. The default tolerance is 0.5 %, half a unit in the third significant figure that published examples print, and the tests write every comparison to a CSV file from which the tables and figures of the companion report [7] and of this section are generated.

8.2 Results

Table 3 summarises the 470 comparisons over 229 cases by suite, and Figure 7 shows every error. Of the 470, 331 are printed values, 118 are formulas of the cited sources and 21 are independent solutions. In total 430 agree within 0.5 % and 449 within 1 %, and the median absolute error is below 0.002 % because many quantities, such as the 107 two-way slab coefficients, the statically determinate frames and the Wood–Armer moments, are reproduced exactly. The 21 comparisons beyond 1 % fall into four understood groups. Nine are the coarse meshes of a plate convergence study, whose finest mesh lies within 0.9 % of the Kirchhoff solution. Two are biaxial column capacities by Bresler's load contour, 8.2–8.4 % below the exact skewed-axis capacity, which quantifies the conservatism of the method. Six are concrete quantities that inherit the rounded SI constants of ACI 318, such as the 2.4 % in ϕV_c, and four are third-mode wall frequencies about 1 % low because Pondus's piers deform in shear, which the Euler–Bernoulli references leave out.

Two of the suites deserve a picture. For moving loads, the unfactored live-load moments per lane of the FHWA two-span continuous steel girder bridge [21] under the HL-93 loading [22] are reproduced within 0.16 % for the vehicle envelopes and within 0.65 % with the lane load (Figure 8). For shear walls, the

Table 3: Calibration of Pondus 0.1.1 by suite: number of cases and comparisons, comparisons within 0.5 % and 1 %, and the median and largest absolute relative errors.

Suite	Cases	Rows	≤0.5 %	≤1 %	Median (%)	Max (%)
Analysis (statics to staged)	63	119	103	111	0.002	17.05
Shear walls, dynamics	9	34	24	30	0.212	1.12
Bridge, HL-93 (FHWA 2A)	1	40	39	40	0.045	0.65
Steel and Eurocode design	21	72	71	72	0.036	0.60
Concrete, timber, masonry	135	205	193	196	0.000	8.40
All	229	470	430	449	0.002	17.05

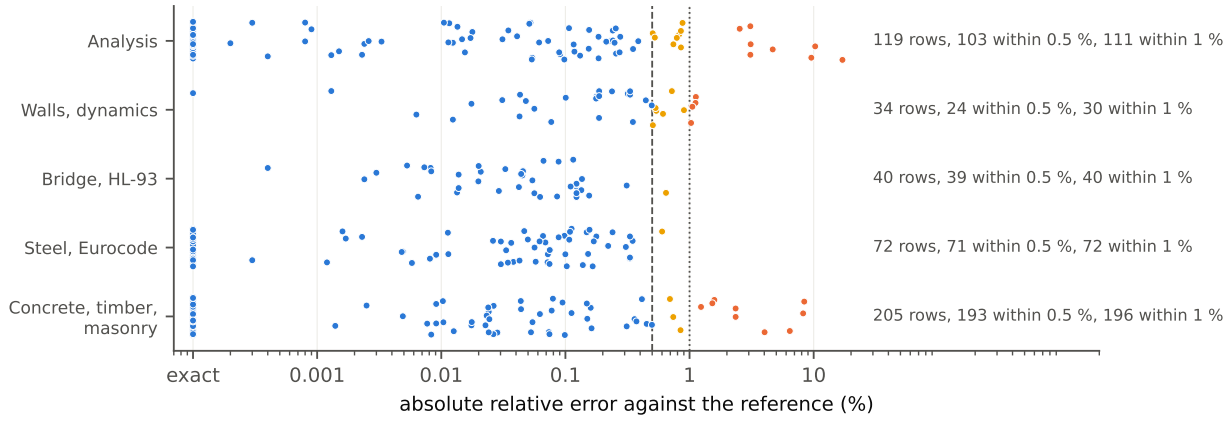


Figure 7: Absolute relative error of all 470 calibration comparisons by suite, on a logarithmic scale; exact agreements are drawn at the left. The dashed and dotted lines mark 0.5 % and 1 %, and the points are coloured blue within 0.5 %, yellow within 1 % and orange beyond.

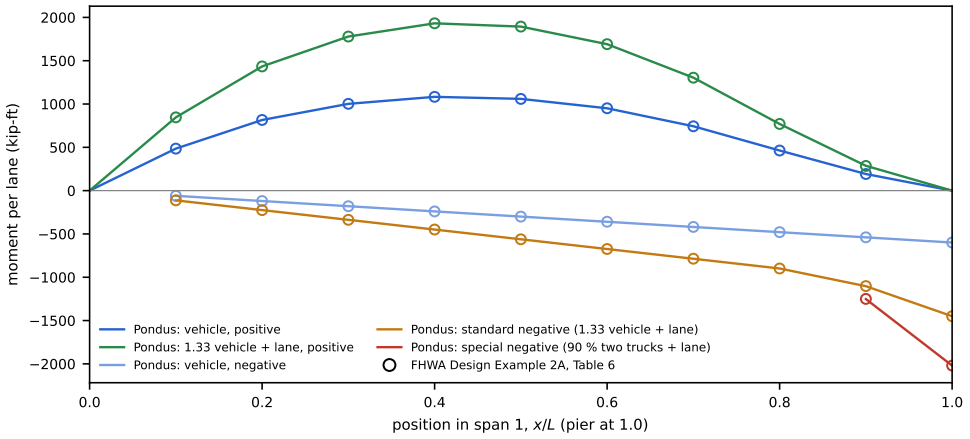


Figure 8: Live-load moments per lane in span 1 of the FHWA two-span bridge under HL-93 loading: Pondus (lines) against Table 6 of the design example (circles).

cantilever frequencies of Clough and Penzien [23], the flexural–shear continuum of a wall coupled to frames [24, 25] and a one-storey system with torsional coupling [26] are reproduced within 0.33 % in the first frequency and within 1.12 % in the higher modes. Beyond these calibrations, which have exact references, a validation against the full-scale seven-storey wall building slice tested on the UCSD–NEES shake table [27–30] gives the measured first period within 6.2 % and the second-mode frequency within 0.9 % with gross sections, and the ACI 318 effective stiffnesses bracket the periods measured after the earthquake tests (Figure 9).

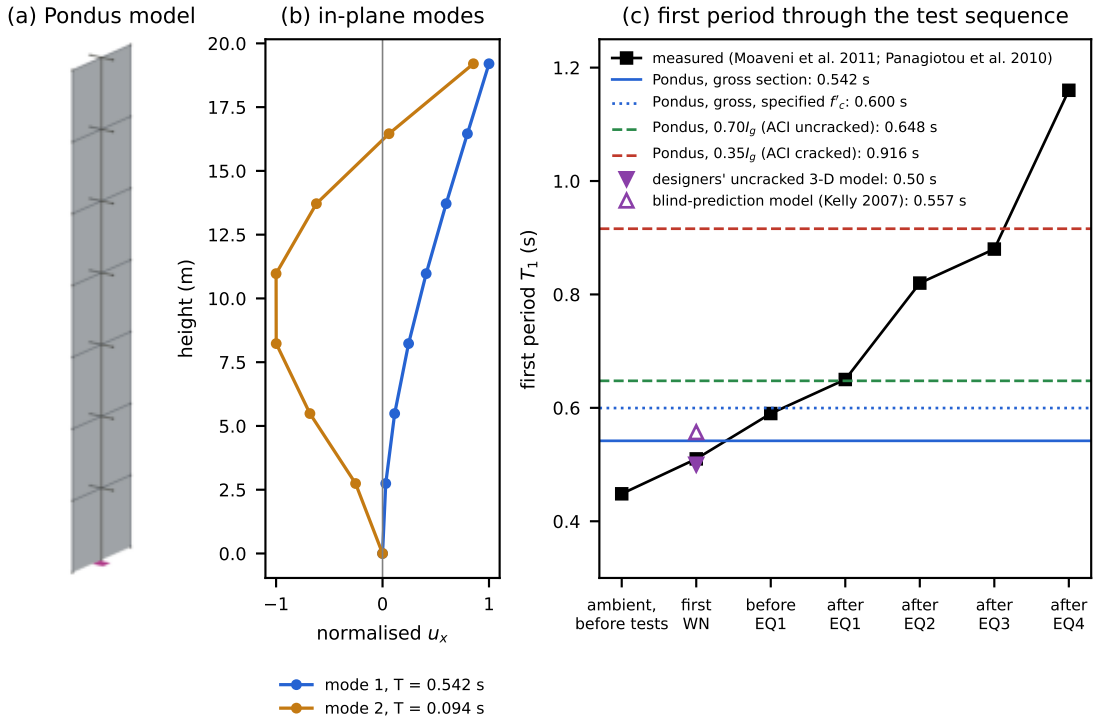


Figure 9: The UCSD–NEES seven-storey wall building slice in Pondus: (a) the model, (b) the first two in-plane mode shapes, and (c) the first period identified at each stage of the shake-table test against Pondus with four stiffness assumptions and two published models.

8.3 Defects found

The calibration, with the closed-form tests that preceded it, found fourteen defects, and Table 4 lists them with their effect before the fix. Six were unconservative, including an equivalent lateral force coefficient 41 % low for stiff structures, a stirrup shear resistance 66 % high in the Eurocode cross-check and a tension rupture capacity 42 % high that ignored the net area and shear lag. Thirteen of the fourteen sat in the code that decides what a kernel or a check is asked, such as which axis a foundation modulus belongs to, which branch of a spectrum a period falls on, which length the minor axis buckles over or how stiff a rigid arm may be made before the solver loses precision. Only one, the sign in the Winkler foundation matrix, was an error of a numerical kernel. Every defect was fixed and the whole suite was run again after each fix, and the suite now runs with every change to the program.

8.4 Calibration against the SAP2000 and ETABS verification manuals

A second calibration used the verification problems that Computers and Structures, Inc. publishes for its own programs, because these are the benchmarks against which practising engineers judge commercial software.

Table 4: Defects found by the calibration and the closed-form tests, with their effect before the fix. Unconservative effects are marked †.

No.	Defect	Where	Effect before the fix
1	ELF C_s read from the ascending branch of the spectrum below T_0	seismic	base shear 41 % low†
2	moving-load envelope from element ends only	engine, wrapped	absolute maximum moment 1.2 % low†
3	Winkler moduli passed as $[k_y, k_z]$ instead of $[k_z, k_y]$	analysis wrapper	vertical foundation missing
4	one effective length for both axes	AISC, EC3	$\phi_c P_n$ 36 % low
5	EC3 class from the pure-compression web limit	EC3 wrapper	class 4 instead of class 1
6	EC3 C_2 left equal to C_1 for a top-flange load	EC3 wrapper	M_{cr} 31 % low
7	EC2 cross-check took the stirrups at the main-bar grade	EC2 wrapper	$V_{Rd,s}$ 66 % high†
8	tension rupture on the gross area	AISC	rupture capacity 42 % high†
9	first interior support of a two-span one-way slab at $wl^2/10$	slab coefficients	moment 10 % low†
10	one thermal coefficient for every material	engine default	restraint forces in concrete 20 % high
11	staged member forces from Ku without fixed-end forces	engine	end moment 1.6 % low†
12	sign of the w–θ_y coupling in the Winkler matrix	engine kernel	uneven settlement of a uniform beam
13	rigid wall arms of absolute stiffness	wall generation	40-storey wall-frame f_1 11.5 % high, drift 21 % low
14	moving envelope at a point read one element away	engine, wrapped	bridge M^+ 0.8 % high

The SAP2000 verification manual prints each analysis problem as a separate document with complete input data, an independent result from a textbook, a hand calculation or a published program, and SAP2000's own result [31]. The ETABS manual adds analysis problems on multistorey frames and walls and design examples to ACI 318-14, AISC 360-10, EN 1992-1-1 and EN 1993-1-1 [32]. Of the 30 SAP2000 frame problems, the 15 ETABS analysis problems and the ACI, AISC and Eurocode design examples, 28 could be modelled with Pondus's features (16 SAP2000 analysis problems, four further ETABS analysis problems and eight ETABS design examples; ETABS Examples 5 to 8 repeat SAP2000 structures), and the others need skewed supports, shear and axial end releases, end offsets, nonlinear links, dampers, isolators, prestress, large displacements or wall sections that Pondus does not have. STAAD.Pro's verification manual is available only as unofficial copies and was not used. Each problem was rebuilt as a Pondus model or design call and compared with the independent value, while the SAP2000 or ETABS value was recorded beside it without being asserted.

Table 5 summarises the 199 comparisons over 64 cases, and Figure 10 shows every error. In all, 177 agree within 0.5 % and 180 within 1 %. The static frame problems, the no-tension and no-compression members, the beam on an elastic foundation, the portal buckling load, the staged cantilever and the shear, axial and bending deformations of SAP2000 Example 1-018 are reproduced exactly, and the periods and response spectrum results of the eight modal problems, including the L-shaped braced building of SAP2000 1-025 with torsional coupling and the seven-storey frame under the El Centro spectrum, agree within 0.26 % apart from one eigenvalue discussed below. The design examples agree within 0.3 % wherever the editions coincide, and the ACI shear terms carry the known offsets of the SI constants of ACI 318-19 against the psi constants of ACI 318-14.

The 19 comparisons beyond 1 % have known causes. Nine are the planar walls of ETABS Example 15a, whose reference is a refined plane-stress mesh in SAP2000: Pondus's wide-column wall is a Timoshenko beam and is 0.8–3.7 % more flexible on walls of three and six storeys and 3.3–6.6 % on squat one-storey

Table 5: Calibration of Pondus against the CSI verification manuals for SAP2000 and ETABS, by group: cases, comparisons, comparisons within 0.5 % and 1 %, and the median and largest absolute errors against the independent values.

Group	Cases	Rows	≤0.5 %	≤1 %	Median (%)	Max (%)
Frame statics, nonlinear members, foundation	29	67	67	67	0.000	0.06
Second order, stability, staged construction	5	10	9	9	0.001	1.75
Modal and response spectrum	13	63	62	62	0.05	1.10
Shear walls (against SAP2000's refined mesh)	9	9	0	1	2.76	6.63
Design (ACI 318, AISC 360, EN 1992, EN 1993)	8	50	39	41	0.02	2.37
All	64	199	177	180	0.02	6.63

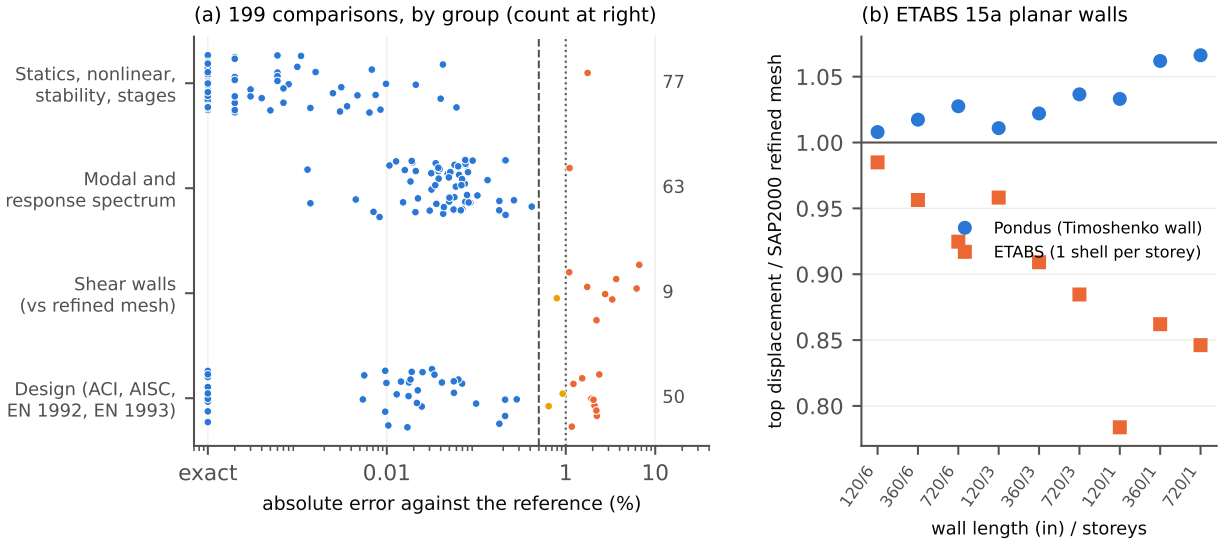


Figure 10: Pondus against the CSI verification manuals. (a) Absolute error of the 199 comparisons by group, with the 0.5 % and 1 % lines. (b) Top displacement of the nine planar walls of ETABS Example 15a divided by SAP2000's refined plane-stress mesh, for Pondus and for ETABS's own wall shells.

walls, where plane-stress effects at the fixed base stiffen the mesh. ETABS's own walls of one shell per storey lie 1.5–22 % below the same reference, so Pondus is closer to it in all nine cases (Figure 10b). Five are the ACI 318-14 wall example, whose hand iteration is not converged, because at its neutral axis depth of 30.1 in its two equilibrium equations give 1035 and 1026 kips. Pondus finds a demand/capacity ratio of 1.009 on the stated eccentricity, close to the 1.007 that ETABS prints. Four are the SI constants of ACI 318. The third eigenvalue of the Bathe–Wilson frame is 1.1 % high, because the references lump the mass at the joints of one element per member whereas Pondus's consistent mass is converged, and the difference grows with the mode number from 0.05 % to 1.1 %. The last is a finding about Pondus itself: in the P-Delta analysis of a tension-stiffened beam the mid-span deflection is exact, but the moment is 1.8 % high at the default eight elements per member and within 0.05 % at 128, because the moments of cubic elements with geometric stiffness converge with the square of the element length.

Table 6: Findings of the CSI calibration and the changes made to Pondus. Unconservative effects are marked †.

No.	Finding	Effect and change
15	EN 1993-1-1 Annex B interaction took $C_{my} = C_{mz} = C_{mLT} = 1.0$	interaction 1.233 instead of 0.873 (+41 %); Table B.3 from the moment diagram
16	EN 1993-1-1 6.2.8, the moment resistance reduced for high shear, was not applied	$M_{y,V,Rd}$ 8.4 % high†; moduli reduced by $\rho A_w^2/4t_w$
17	EN 1993-1-1 6.2.9.1, the plastic N + M section check of Class 1 and 2 I sections, was missing	possibly unconservative for short members†; check added
18	no lumped nodal masses, no SRSS combination, no frame shear deformation	the storey-mass, SRSS and shear problems not reproducible; all three added
19	a node stabilised only by a diaphragm made the model fail, because the unconstrained stiffness was factorised for every model	robustness; skipped for diaphragm models
20	a node joined only by axial members had free rotations, and an all-truss model with diaphragms lost its in-plane rotation	robustness; rotations held, pinned frames used
21	the engine's diaphragm leaves a slave's rotation about z free, so a mass node as a slave had free rotary inertia	robustness; the mass node is made the master
22	the engine's modal fallback needed a positive definite mass matrix	robustness; semidefinite dense solver (engine change)

The CSI problems also found defects, listed in Table 6 after the fourteen of Table 4. Three concern the Eurocode 3 checks: Pondus took the equivalent uniform moment factors as 1.0 instead of evaluating Table B.3, which was conservative by up to 41 %, and it neither reduced the moment resistance for high shear nor checked the plastic interaction of axial force and moment in the cross-section, the first of which overstated the moment resistance of the 406×178×74 UB of EN 3-2005 Example 002 by 8.4 %. Four are robustness defects that only appeared with the modelling devices that SAP2000 and ETABS users take for granted, namely a storey mass at the centre of mass, a braced building made of truss members only, and the semidefinite mass matrix that results. Every finding was fixed and the whole test suite, including the 470 earlier comparisons, passes after the changes. The fixes added lumped nodal masses, the SRSS combination and an optional frame shear deformation to Pondus, and two marked changes to the engine.

9 Discussion

The two kinds of evidence in this paper point the same way. The two calibrations show that the tools of an agentic program can be made accurate, and that the defects worth finding lie in the interfaces between the model and the kernels, where a wrong axis, a wrong branch or an unsafe default produces a plausible number that neither an agent nor a self-written test is likely to question. The agent runs show that a low-cost model

can drive those tools to a complete, balanced and designed building, with a calculation report, in two to six minutes, and that the numbers it repeats are the tools' numbers. They also show that the prose an agent writes around those numbers is the weak link. Four of the ten audited statements were wrong in their description, one claimed that work had not been done when it had, and the one engineering observation that mattered most, that the wall factors did not suit the frame direction, was not made.

These observations suggest how a program for agents should be built. The answer an engineer acts on should be the program's own output, such as the design tables, the report and the model, rather than the agent's summary, and Pondus is arranged so that every agent run ends in a model and a report that the engineer opens in the ordinary interface. Results that matter for judgement should be returned as explicit checks rather than as raw data, because the agent reliably repeated checks that the tools stated, such as the load balance and the drift ratios, but did not derive one that the tools left implicit, the mismatch between the lateral system and the seismic factors in one direction. Settings that an agent changes during a run, such as the vibration screen in run 2, should be listed by the program in its own output, so that the engineer does not rely on the agent to mention them. The program and the prompt already apply the first principle, and the second and third are improvements that this study identified.

The study has clear limits. Two runs with one language model are a demonstration, not a measurement of agent reliability, and a stronger model would probably make fewer descriptive errors, while language models are not deterministic, so a repeated run may build or describe the same building differently. The audit covered the statements in the answers and the quantities that could be checked by hand, not every bar of every member. The calibration covers the program's tools against references with exact answers and a single shake-table building, and it does not cover the agent. The program itself has limits that its documentation lists, including linear first-order analysis by default, braced-frame column design unless P-Delta is requested, no wind load generator, one set of seismic factors for both directions, no accidental torsion and no irregularity checks, and pushover results that are indicative because the hinges carry no axial interaction.

10 Conclusions

Pondus shows one way to build structural analysis and design software for language-model agents. The agent and the engineer share one model and one set of commands, the agent reaches the program through 44 MCP tools whose descriptions and results are written for an agent, and the built-in chat runs each request on a copy of the model that one Undo can take back. The analysis uses the Stabileo engine as a library, with the model translated into the engine's input at every analysis and four marked changes to the engine, and models pass to and from the Stabileo editor and the Aedifex BIM modeller through Stabileo's file format, which reproduced a frame exactly in a round trip and a wall building within 0.05 %. A low-cost agent built, analysed and designed a two-storey frame in 105 s and a five-storey wall-frame building with an earthquake analysis and a calculation report in 382 s, and every number it reported came from the tools and agreed with hand checks, whereas four of its descriptions were wrong and it did not question the wall factors applied to the frame direction. The calibration against 470 reference values found 430 within 0.5 % and 449 within 1 %, with every larger difference explained, and it found fourteen defects of which thirteen lay in the layer between the model and the numerical kernels. The verification problems of SAP2000 and ETABS gave 177 of 199 within 0.5 %, put Pondus's walls closer to a refined plane-stress mesh than ETABS's own walls, and found eight more defects, four of them robustness failures that appeared only with the modelling devices that users of those programs take for granted. For engineering practice, the results mean that the program's tables and report, which the calibration supports, should be what the engineer checks and signs, while the agent's summary should be treated as a guide to them and never as a substitute.

Declaration of AI use

Pondus and its test suite were coded with the help of the agentic coding tool Claude Code (Anthropic), which also helped draft this paper and its figure scripts. The agent runs of Section 7 used the model `deepseek-v4.1-flash` through OpenRouter. The author designed the program, the calibration and the case studies, checked the results and takes full responsibility for the content of this paper.

Data and code availability

Pondus is free software under the GNU Affero General Public License, version 3 only, and its complete source, including the vendored Stabileo engine with its marked changes and the TigrimOS harness, is available at https://github.com/Sompote/pondus; the results of Sections 2 to 8 before Section 8.4 were produced by version 0.1.1, and the CSI calibration by the development version that includes its fixes. The calibration is reproduced by the tests `calib_analysis`, `calib_steel`, `calib_concrete`, `calib_walls`, `calib_bridge`, `calib_csi_analysis` and `calib_csi_design` of `pondus-core` with `PONDUS_CALIB` set to an output folder; the CSI manuals, the data extracted from them and the notes are in `calibration/sources/csi/` and `calibration/NOTES_csi.md`. The agent runs, their progress logs, prompts, answers, saved models and reports, the Stabileo round-trip files and the scripts that drive the runs and draw the figures are in the folder of this paper (`case/` and `scripts/`).

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[3] Anthropic. Model Context Protocol specification. https://modelcontextprotocol.io, 2024.

[4] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[5] Sompote Youwai. Pondus 0.1.1: agentic structural analysis and design. https://github.com/Sompote/pondus, 2026.

[6] LambdaClass. Stabileo: structural analysis engine and editor (dedaliano-engine), commit 7985e8c. https://github.com/lambdaclass/stabileo, 2026.

[7] Sompote Youwai. Calibrating an agentic structural analysis and design program against published benchmarks. Pondus repository, paper/paper.pdf, 2026.

[8] Sompote Youwai. TigrimOSR 0.7.2: a native agent platform and command-line agent (tigrim). https://github.com/Sompote/TigrimOSR, 2026.

[9] E. N. Dvorkin and K. J. Bathe. A continuum mechanics based four-node shell element for general non-linear analysis. *Engineering Computations*, 1(1):77–88, 1984.

[10] B. Stafford Smith and A. Coull. *Tall Building Structures: Analysis and Design*. Wiley, New York, 1991.

[11] ASCE. *Minimum Design Loads and Associated Criteria for Buildings and Other Structures (ASCE/SEI 7-22)*. American Society of Civil Engineers, Reston, VA, 2022.

[12] Department of Public Works and Town & Country Planning. *Seismic Resistant Design of Buildings and Structures (DPT 1301/1302-61)*. Ministry of Interior, Bangkok, 2018.

[13] A. Der Kiureghian. A response spectrum method for random vibration analysis of MDF systems. *Earthquake Engineering and Structural Dynamics*, 9(5):419–435, 1981.

[14] ACI Committee 318. *Building Code Requirements for Structural Concrete (ACI 318-19) and Commentary*. American Concrete Institute, Farmington Hills, MI, 2019.

[15] ACI Committee 318. *Building Code Requirements for Reinforced Concrete (ACI 318-63)*. American Concrete Institute, Detroit, 1963.

[16] R. H. Wood. The reinforcement of slabs in accordance with a pre-determined field of moments. *Concrete*, 2(2):69–76, 1968.

[17] G. S. T. Armer. Discussion of: The reinforcement of slabs in accordance with a pre-determined field of moments. *Concrete*, 2(8):319–320, 1968.

[18] B. Bresler. Design criteria for reinforced columns under axial load and biaxial bending. *Journal of the American Concrete Institute*, 57(11):481–490, 1960.

[19] AISC. *Specification for Structural Steel Buildings (ANSI/AISC 360-16)*. American Institute of Steel Construction, Chicago, 2016.

[20] CEN. *Eurocode 3: Design of steel structures, Part 1-1: General rules and rules for buildings (EN 1993-1-1)*. European Committee for Standardization, Brussels, 2005.

[21] Federal Highway Administration. Steel Bridge Design Handbook Design Example 2A: Two-Span Continuous Straight Composite Steel I-Girder Bridge. Technical Report FHWA-HIF-16-002 Vol. 21, U.S. Department of Transportation, Washington, DC, 2015.

[22] AASHTO. *AASHTO LRFD Bridge Design Specifications*. American Association of State Highway and Transportation Officials, Washington, DC, 7th edition, 2014.

[23] R. W. Clough and J. Penzien. *Dynamics of Structures*. Computers and Structures, Berkeley, 3rd edition, 2003.

[24] A. C. Heidebrecht and B. Stafford Smith. Approximate analysis of tall wall-frame structures. *Journal of the Structural Division, ASCE*, 99(ST2):199–221, 1973.

[25] E. Miranda and S. Taghavi. Approximate floor acceleration demands in multistory buildings. I: Formulation. *Journal of Structural Engineering*, 131(2):203–211, 2005.

[26] A. K. Chopra. *Dynamics of Structures: Theory and Applications to Earthquake Engineering*. Pearson, 5th edition, 2017.

[27] M. Panagiotou, J. I. Restrepo, J. P. Conte, and R. E. Englekirk. The Design and Shake Table Testing of a Full-Scale 7-Storey Reinforced Concrete Cantilever Wall. In *Proceedings of the NZSEE Annual Technical Conference*, Wellington, 2010. Paper 03.

[28] M. Panagiotou, J. I. Restrepo, and J. P. Conte. Shake-Table Test of a Full-Scale 7-Story Building Slice. Phase I: Rectangular Wall. *Journal of Structural Engineering*, 137(6):691–704, 2011.

[29] B. Moaveni, X. He, J. P. Conte, J. I. Restrepo, and M. Panagiotou. System Identification Study of a 7-Story Full-Scale Building Slice Tested on the UCSD-NEES Shake Table. *Journal of Structural Engineering*, 137(6):705–717, 2011.

[30] T. E. Kelly. A Blind Prediction Test of Nonlinear Analysis Procedures for Reinforced Concrete Shear Walls. *Bulletin of the New Zealand Society for Earthquake Engineering*, 40(3):142–159, 2007.

[31] Computers and Structures, Inc. *SAP2000 Software Verification: Examples 1-001 to 1-030 and 2-001 to 2-020*, 2016. https://docs.csiamerica.com/manuals/sap2000/Verification/, accessed 22 October 2026.

[32] Computers and Structures, Inc. *Software Verification Examples for ETABS 2016*, 2017. ISO ETA122815M6 Rev. 1, https://docs.csiamerica.com/manuals/etabs/SoftwareVerification.pdf, accessed 22 October 2026.